Breaking the Validator Stall Loop

Cost-accounted order-then-execute, built on the cost-accounting papers: analysis, design, throughput model, and development plan

F1R3FLY.io — design document, revision 4

October 29, 2026

Repository	`F1R3FLY-io/f1r3node-rust`
Code basis	`integration/cost-accounted-ote-b1d866e45` at `d1f3ab5e1` (Oct 28, 2026), which carries Track A and the B4 work through patch 0035
Design basis	The cost-accounting papers in `F1R3FLY-io/publications` at `37915af` (Oct 28, 2026), read directly (§B)
Revision 4	Records the agreed relationship between REV and the token stacks. Wallets are disaggregated from the vault: REV lives in located stacks on its owners' channels, and moving REV is a native operation. Phlo is what a COMM spends, minted per deployment at the deployment's stated price. The system is linear: REV changes hands, and is never created or destroyed by computation. Consumed REV moves to the proposer that the block names. Revision 3 treated the vault as permanent and REV payments as a later track; that part is withdrawn.
Revision 3	Rebuilt the cost-accounting design on the papers and added a measured throughput model. The stall analysis, the consensus seam and the finalizer design of revision 2 stand.
Method	Code reading, and measurements on a 2-vCPU host (Intel Xeon at 2.1 GHz, release build). Absolute times are host-specific. The ratios carry the argument.

Abstract

Validators stall as they come under sustained load. The stall analysis of revisions 1 and 2 stands: a feedback loop through the finality lag, closed by two hard limits, with run-to-completion execution and the merge it forces at its root. This revision builds the cost-accounting half of the design on the papers. Wallets are disaggregated from the vault: each wallet is a stack of REV located on its owner's own channel, so moving REV is a native operation on two channels, with no shared contract on its path. A deployment states a phlo limit and a phlo price. A validator accepts or rejects it on that price. Its precharge moves REV from the wallet into an escrow and mints phlo against it. Each COMM spends phlo. Settlement moves the REV of the consumed phlo to the proposer, and returns the rest to the wallet. The system is linear: every unit of work happens because REV changes hands, and nothing is burned. The implementation today keeps money in the SystemVault, reads it through Rholang contracts, and settles every deploy through one shared vault contract that burns the charge. Measured on the integration branch, settlement takes 83–94 % of block execution, and a one-line deploy spends 1.2 ms interpreting its term and about 675 ms on funding and settlement. The throughput model projects a 12–105-fold

reduction of per-envelope work on the measured workloads, about 100-fold for REV transfers, and near-linear scaling with cores for envelopes with independent signers. That is two to three orders of magnitude over today's measured 0.44–1.24 envelopes per second for small deploys and transfers, and one to two orders for heavier deploys. The model also identifies where the gains stop: deploys dominated by their own computation, application-level hot channels, the consensus cap of envelopes per block, and multi-core RSpace efficiency, which is not yet measured.

Contents

1 Summary **4**

2 What revisions 3 and 4 change, and why **5**

3 The stall: analysis **6**

3.1 The core loop 6

3.2 Costs that grow with the lag, and the starvation paths 6

3.3 Soak evidence 7

4 The blueprint: what the papers specify **7**

4.1 Signatures are names; token stacks are located terms 7

4.2 Payment is the communication 7

4.3 Acceptance by linear proof is the concurrency control 7

4.4 Overcharge and refund 8

4.5 Located purses make the proof local 8

4.6 Chains and antichains: where hot spots come from 8

4.7 Fees, minting, sponsors and multi-signature 8

4.8 Wallets disaggregated from the vault 9

4.9 Linearity: work happens because REV changes hands 9

4.10 Internalisation 10

5 The gap, measured **10**

5.1 How the funding path works today 10

5.2 Measurements 11

6 Design: cost-accounted order-then-execute on REV stacks **12**

6.1 Principles 12

6.2 Price and acceptance 12

6.3 The life of an envelope 13

6.4 REV transfers 13

6.5 Supply 14

6.6 Owners, sponsors and grants . . . 14
6.7 Block execution . . . 14
6.8 Records . . . 14
6.9 What leaves the v6 path . . . 14
6.10 The consensus seam (unchanged from revision 2) . . . 15

7 Throughput analysis **15**
7.1 Model . . . 15
7.2 Per-envelope work . . . 16
7.3 Node throughput . . . 16
7.4 Where the anticipated gains do not appear . . . 17
7.5 Validation . . . 17

8 Development plan **18**
8.1 Status (October 29, 2026) . . . 18
8.2 Track B, revised: REV stacks and native settlement . . . 18
8.3 Other tracks . . . 19

9 Decisions, risks and open questions **20**
9.1 Decisions requested . . . 20
9.2 Risks . . . 20
9.3 Open questions . . . 21

A Detailed stall findings **21**

B Sources **23**

1 Summary

The stall. Under sustained deploy load the unfinalized region of the DAG grows. Proposal preparation, merging, the deploy lifecycle register and the finality oracle all do work that grows with that region, which slows blocks and finality, which grows the region. Two limits turn the slowdown into a stop: a 15-second finalizer timeout that discards all work, and a merge cap at 256 blocks of floor distance (§3). Track A has delivered the fixes for the finality half of the loop on `dev`.

The cause underneath. Each deploy runs to completion against committed state, so concurrently produced blocks hold divergent post-states, and the node must merge them. The cost-accounting papers remove the reason for the merge: every interaction is gated by a token of the signatures involved, so potential conflicts are visible as shared signatures, and deploys can run at any time.

The two central points of the design.

1. **No hot spots.** Wallets are disaggregated from the vault. Each wallet is a stack of REV located on its owner's own channel. A deploy touches only the wallets and stacks that its own signatures name. Two deploys with different signers touch disjoint state and cannot contend. A shared structure on the payment path would put every deploy on one causal chain, and that is what the papers rule out.
2. **Concurrent deploys, little merge logic.** Acceptance by linear proof is the concurrency control. Competing deploys are competing proof obligations over the same linear hypotheses, and at most one succeeds before any step runs. Independent deploys then run concurrently. The only conflicts left are races on shared data channels, which the runner decides and records.

REV and phlo. REV is the quantity held in wallets. Phlo is what a COMM spends. A deployment states a `phloLimit` and a `phloPrice`, and a validator accepts or rejects it on that price. At acceptance, REV for the overestimate moves from the deployer's wallet into an escrow, and phlo is minted against it at the deployment's price. Each COMM spends phlo. At settlement, the REV of the consumed phlo moves to the proposer's channel, and the rest returns to the wallet. The system is linear: work happens because REV changes hands, REV is never created or destroyed by computation, and the counterparty of every payment is a name in the term that the block fixes (§4.9).

Where the implementation stands. The consensus seam is largely in place: order-then-execute, structural validity, the linearizer, records, attestation, inclusion checks and pruning (patches 0028–0035). The funding path departs from the papers. Money stays in the SystemVault. Each deploy reads vault balances through Rholang (44.5 ms per read), and after its run one vault contract reserves, validates, apportions, burns the charge, pays a shared fee vault and refunds. That play runs with an unlimited budget and writes about 1,000 events into the record. A user REV transfer through the vault costs 81.6 ms of contract code on its own. Every settlement touches the same vault map, fee vault and cost cursors, so settlements form one causal chain. The measured consequence: settlement is 83–94 % of block execution, and the branch executes 0.44–1.24 envelopes per second on the measurement host (§5).

Projected throughput. Per-envelope work falls 12- to 105-fold on the measured workloads and about 100-fold for REV transfers, and independent envelopes scale with cores. For one-line deploys and transfers the projection is about 160–280 envelopes per second on the 2-vCPU measurement host and 1,300–2,250 on 16 cores, against 0.44–1.24 today. Where gains are smaller, §7.4 explains why, item by item.

Decisions requested (§9.1).

(1) Wallets are located REV stacks on their owners' channels, disaggregated from the vault. Moving REV is native.

(2) Phlo is minted per deployment at its stated price. The shard minimum price is a validity floor, and each validator sets its own acceptance price above it.

(3) Consumed REV moves to the proposer that the block names. Nothing is burned, and there is no fixed fee apart from phlo.

(4) Owners, sponsors and grants fund through REV stacks and funding slots.

(5) The throughput targets of §7.5 become exit criteria for B3 and B7.

2 What revisions 3 and 4 change, and why

Revision 2 stated in its sources that it "did not read the papers directly", and took them as the branch's proposal quoted them. It got the signature-marking argument right: different signatures cannot conflict on funding, and agreement reduces to the race winners of each block and the order of conflicting blocks. It missed what the papers say about money. It does not mention token stacks. It adopted the branch's settlement as a library to port. It kept the vault's contention patch as a requirement ("`SystemVault.rho` bucket locks (DR-119): Keep"). It defined the record to include "settlements, system work", so the vault's event logs entered consensus.

The work of the week of October 26 traced each of these to a defect:

- **Record size.** Records came to about 100 KB per envelope, 93 % logs, and 92 KB of that was the vault settlement log. A block of more than about 80 envelopes produced a record that no block could carry (patch 0035).
- **Unbounded system work.** Settlement runs with `Cost::unsafe_max()`, the gate's estimator cannot bound the vault contracts, and system plays never draw on the meter: a test measured 0 phlo under any budget.
- **The vault as a hot spot.** A precharge-and-refund design built on the vault measured worse than the code it replaced: 118 KB per one-source send. That design is withdrawn.

Revision 3 started from the papers and measured the gap. It still treated the vault as the permanent custodian of REV, with tokens bought from it at a boundary, and it put REV payments in a later track. Revision 4 corrects that. The papers' abstraction is a disaggregation of wallets from the vault: REV itself lives in located stacks, phlo is minted against it per deployment, and every payment is REV changing hands in a linear system.

Everything in revision 2 that does not depend on the funding path stands: the stall analysis, the consensus seam, the canonical order, records and attestation, the finalizer design, backpressure, Track A and the hook points.

3 The stall: analysis

This section restates revision 2 in brief. The evidence and line references are unchanged, and Appendix A lists the detailed findings.

3.1 The core loop

Under sustained load three quantities grow together: the pending deploy pool, the set of deploys included in blocks but not yet settled, and the unfinalized DAG cone. Each grows with the finality lag D, and each carries costs that scale with it (Figure 1).

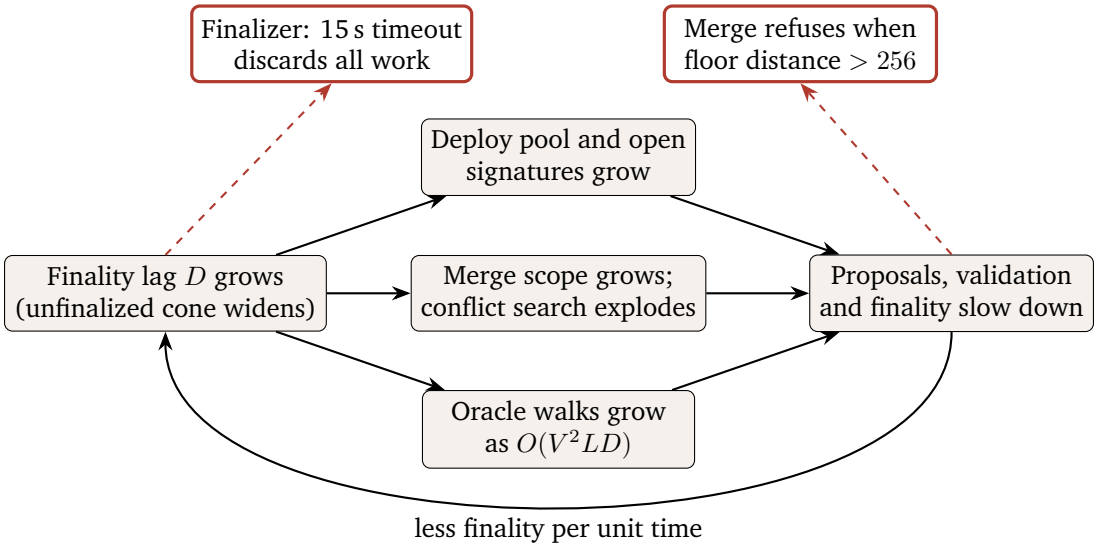


Figure 1: The feedback loop. Every cost in the middle column grows with the finality lag. The two traps (outlined) turn the slowdown into a stop.

3.2 Costs that grow with the lag, and the starvation paths

- **Deploy pool.** About $8N$ signature checks per proposal for a pool of N, re-read under a lock.
- **Lineage walks.** Quadratic in load: open signatures times segment length times deploys per block.
- **Rejection-option search.** Exponential in conflicting pairs per group. Vault and PoS channels collapse groups into one, so the vault is a hot spot for the merge as well.
- **Hot channels in RSpace.** $O(n)$ per touch and $O(n^2)$ over a run for a channel that accumulates n datums. Replay scans its COMM list with a copy per step, $O(n^2)$ in COMMs (clone audit, finding 0.2).
- **Finality walks.** $O(P \cdot V^2LD)$ warm per block.
- **Starvation.** A serial proposer queue, synchronous locks in async code, and CPU-bound consensus work on the async workers that also serve the network.

3.3 Soak evidence

The branch's proposal records two soaks. In August one merge call averaged 2.16 s, 92 % of it settled-signature probes. In October no single stage separated failing sessions from passing ones, and the binding constraint had moved toward finality and Casper bookkeeping. The production high-phase replay targets recorded in `cost_accounting_perf_spec.rs` attribute about 229 ms per deploy to precharge, 189 ms to refund and 32 ms to evaluation. That is, 93 % of replay time per deploy is spent moving money through the vault.

4 The blueprint: what the papers specify

This section reads the design from the papers. Each point ends with its consequence for the implementation.

4.1 Signatures are names; token stacks are located terms

The cost-accounted rho calculus extends names to $x, y ::= \lceil T \rceil \mid s$: signatures are channel names. Token stacks $S ::= () \mid s : S$ are first-class terms. Each layer is a balance cell of one signature. The companion paper refines the stack to a *located* stack $S(I, s : S')$, a stack indexed by the surface I that may draw on it. In rho this is a token stack sitting on a specific channel: only processes holding a matching channel name can consume from it. Unforgeable channels make that a genuine capability.

Consequence. A signer's funds for computation are a stack on the signer's own channel in RSpace. They are not a balance in a shared contract.

4.2 Payment is the communication

The five rewrite rules are the COMM rule gated by token consumption. For a single signature,

$$\{\, \texttt{for}(y \leftarrow x)\, T \mid x!(U)\, \}_s \;\mid\; s : S \;\longrightarrow\; T\{\lceil U \rceil / y\} \;\mid\; S.$$

The other four rules cover compound signatures and split or combined tokens. Joins follow one schema quantified over partitions, and conservation of authority holds across all of them: a join consumes exactly one receiver authority and one sender authority per channel.

Consequence. The reducer charges by popping cells from located stacks. There is no separate settlement step that converts usage into money after the fact.

4.3 Acceptance by linear proof is the concurrency control

A deployment is accepted if and only if a linear proof of its resource use exists: for every signature s it names, the supply Σ_s meets the demand Δ_s. The check runs before any step executes, and it is linear in the size of the AST. In the papers' words, two deployments competing for the same tokens are "two proof obligations competing for the same linear hypotheses. At most one can succeed." Duplicate and simultaneous arrivals are resolved by the same rule, "without locks, priority queues, or rollback mechanisms".

Consequence. Admission needs only the stacks of the envelope's own signers. Group decisions, later holds and shared cursors are unnecessary: committing tokens to an accepted deployment removes them from the supply that the next proof sees.

4.4 Overcharge and refund

Where control flow is data-dependent, the validator computes a conservative demand $\Delta_c^{\max}$, the maximum over reachable branches, and accepts if $\Sigma_c \geq \Delta_c^{\max}$. Execution consumes only the tokens it forces, and "the residual $\Delta_c^{\max} - \kappa$ is returned to the client's token stack as part of the deployment's settlement". The refund is safe because the number of tokens consumed equals the number of forced redexes, which the runtime counts exactly.

Consequence. Settlement is a return of unused cells to the client's own stack. It is not a vault transaction.

4.5 Located purses make the proof local

Locating stacks decomposes the global resource obligation into one local linear proof per purse. The global proof is the separating conjunction of the local ones (companion paper, "Local sufficiency composes"). Location also quarantines data-dependence to the surfaces that need a conservative bound.

Consequence. The gate checks each signer's stack independently, and independent signers' checks can run concurrently.

4.6 Chains and antichains: where hot spots come from

The spacetime paper maps a cost-accounted computation to a measured causal set. Events are COMM firings, the order is enabling, and concurrency is Bernstein independence of footprints: no write-write and no read-write overlap. Two of its results bear directly on throughput:

- **Time from the stack.** The firings that draw on a single located stack form a chain, ordered by stack depth. A shared stack, or any shared resource that every deploy writes, serializes every deploy that touches it.
- **Space from the surfaces.** Firings that draw on distinct stacks at non-near surfaces are concurrent: they form an antichain.
- **Number is volume.** The phlogiston spent equals the number of events, and discrete proper time is the longest chain.

With W the total work and L the longest chain, a scheduler on P cores completes the work in at most $W/P + L$ (Brent's bound). Throughput therefore depends on keeping L short: no state that every deploy writes.

Consequence. A design meets the papers' throughput promise only if nothing on the payment path is shared across signers. A shared vault map, fee vault or cursor puts every settlement on one chain, and L grows with W.

4.7 Fees, minting, sponsors and multi-signature

- **Fees.** In the worked example, the validator handler takes one token from the client's address A_c and keeps it on its own fee channel F_v. The fee lands on a per-validator channel.
- **Minting.** Minting is constructing a token stack and sending it on a signature channel. Epoch phlogiston for validators, slush funds and fee-to-stake conversion use the same primitive.

- **Funding slots.** A contract creates an unforgeable channel and lets any funder deposit tokens on it. This decouples who authorizes, who pays and what runs. A sponsor funds a new user through a slot.
- **Multi-signature.** A compound signature $s_1 \otimes s_2$ needs tokens from every component. Weakening is forbidden, so a compound token must be spent in full or explicitly split.

Consequence. Sponsors, grants and owners are funding slots and located stacks, not post-run vault debits.

4.8 Wallets disaggregated from the vault

The located stack is how value is held, not only how fuel is held. A wallet is a stack of REV located on its owner's channel. The channel is a function of the owner's signature: a public key, an unforgeable name, or a compound signature for a multi-signature wallet. Finding a wallet is therefore a computation, not a lookup in a shared map. Only a term signed by the owner can draw on the stack. Anyone can deposit to it, as anyone can fund a funding slot.

Cells carry amounts. A balance is a count on a cell, not a list of unit cells, so spending splits a cell and receiving adds one. Deposits land as separate datums on the payee's channel and are combined only when the payee spends. Payments to one payee therefore commute: two deposits never conflict, and only spends from one wallet form a chain.

Consequence. Moving REV is a COMM on the payer's and payee's channels. It needs no registry lookup, no map walk, no purse split through a shared mint and no settlement afterwards.

4.9 Linearity: work happens because REV changes hands

Linear proofs work because the system is linear. Every token is consumed exactly once, and consuming it moves it to someone. Computation never creates or destroys REV. So the destination of consumed REV is not a policy choice: it moves to the party that the term names as the counterparty.

A minimal example: client c holds r REV in its wallet W_c. Its deployment is one COMM, so $\Delta = 1$ phlo, at price p. Validator v proposes the block that includes it.

```
D  =  { x!(5) | for(y <- x) Nil }_c          -- one COMM, signed by c

block b, sender v, denotes at its canonical position:

  W_c!(r)                                    -- c's wallet: r REV
| for(r <- W_c){ W_c!(r - p) | E_D!(p) }     -- precharge: p REV to escrow E_D
| D | c:()                                   -- one phlo cell, minted against E_D

-> Rule 1 (the COMM fires and spends the cell, k = 1):
  { x!(5) | for(y <- x) Nil }_c | c:()  ->  Nil | ()

-> settle (k = 1, Delta - k = 0):
  for(a <- E_D){ F_v!(a) }                   -- p REV from E_D to v's channel
                                             -- refund to W_c: p * (Delta - k) = 0

conservation:  (r - p) at W_c  +  p at F_v  =  r
```

The recipient is a name in the term, F_v, and v is the block's sender. The term is fixed when v proposes the block. Whoever reduces it, the runner or any replaying validator, reaches the same post-state, and appears nowhere in the term. Paying the runner would make executed state depend on who produced the record, not on the ordered blocks. So consumed REV moves to $F_{\mathrm{sender}(b)}$, and running and replaying are the work every validator does as a member of the shard.

Consequence. Conservation is an invariant of every block: the sum over wallets, escrows and validator channels is unchanged. There is no burn and no fixed fee apart from phlo.

4.10 Internalisation

The papers give a source-to-source translation of the cost-accounted calculus into pure Rholang 1.1, with splitter and joiner infrastructure. This gives cost-accounted semantics on today's runtime while the native implementation is built. The history-and-cost note adds that such an internalisation is lax: it carries an overhead grade. The native implementation is therefore the throughput path, and the translation is a bridge.

5 The gap, measured

5.1 How the funding path works today

The branch already has the papers' mechanism, but the v6 path does not use it:

- **Located stacks exist.** The reducer evaluates a token stack term by producing a `CostStack` datum on the signature channel of its head cell (`reduce.rs`, `eval_cost_stack`). The supply module models purse stacks and their inventory (`costacc/supply.rs`). Stack pops are accounted natively (`accounting/authority.rs`, `stack_pops`).
- **v6 offers bypass them.** An owner-direct offer funds "every class of the signer's own principal lane" from the signer's vault (`owner_offer.rs`). The gate's supply "counts SystemVault balances only" (`gate.rs`). Fresh usage is bought from REV at the signed price on every deploy.

Blueprint	Implementation (v6 path)	Measured effect
Supply is the depth of the signer's located stack.	Supply is read from SystemVault balances through Rholang queries.	44.5 ms per balance query. The run phase spends 110–160 ms beyond interpreting the term.
Payment is the COMM popping a cell.	The run is metered natively. The charge is then converted to REV and settled by `applyPhloCost`.	560–600 ms and about 1,000 events per envelope, under `Cost::unsafe_max()`.
Refund returns $\Delta^{\max} - \kappa$ to the client's stack.	`reserveAll`, `validateSettlement` and `settleReserved` run together after the run.	The reserve-and-refund happens, collapsed into one play after the run.

Blueprint	Implementation (v6 path)	Measured effect
The linear proof is the concurrency control.	Holds that move no funds, plus `decide_groups`, `LaterHolds`, a held-capacity guard and the DR-28 cursors.	Settlement runs sequentially in block order.
Consumed REV moves to the counterparty; the system is linear.	`settleReserved` burns the resource charge, leaving it in a purse that nobody can reach. Only the fee moves.	REV is destroyed by computation, which breaks conservation.
A wallet is a located stack; moving REV is a COMM.	Wallets are purses in the vault's tree map, reached by registry lookup; a transfer runs the mint's split and deposit.	81.6 ms of contract code per user transfer.
The fee goes to the validator's own channel.	One fee vault, reached through `findOrCreate` by every settlement.	One write target shared by every envelope.
Distinct signers form an antichain.	Every settlement touches the vault map, the fee vault and the cursor map.	Settlement time per envelope grows with block size (Table 2).
The record carries the COMM log with charges.	The record also carries the wallet and receipt logs of the vault contracts.	About 92 KB of a 100 KB entry.

5.2 Measurements

All figures come from release builds on the 2-vCPU host named on the first page, using tests in `casper/tests/v6_executor_spec.rs` and `casper/tests/util/rholang/cost_accounting_perf_spec.rs`.

Envelopes	Runs	Settlement	Settlement share	Settlement per envelope	Envelopes/s
1	0.14 s	0.67 s	83 %	0.67 s	1.24
4	0.51 s	2.73 s	84 %	0.68 s	1.24
16	1.84 s	14.2 s	88 %	0.89 s	1.00
64	8.13 s	136.6 s	94 %	2.14 s	0.44

Table 2: Block execution on the integration branch by block size (one-line sends from distinct payers; `phase_timings_by_block_size`). Settlement per envelope grows with block size, which is the mark of shared state.

Legacy `dev` pays the same cost. The legacy precharge and refund system deploys, which also move REV through the vault, take 364 ms and 291 ms per deploy on the play path, against 0.3 ms of evaluation. On replay, a block of seven deploys spends 106 ms on precharge, 70 ms on refund and 0.5 ms on evaluation per deploy. The vault-centred payment path is the dominant per-deploy cost on both the legacy and the v6 path.

System work is invisible to the meter. The vault plays measured on October 28 report 0 phlo used

Workload (one envelope)	Interpret term	v6 run phase	v6 settlement	Useful share
One send	1.2 ms	112 ms	563 ms	0.17 %
256 sends	29.7 ms	192 ms	605 ms	3.7 %
300 recursive COMMs	32.5 ms	184 ms	572 ms	4.3 %
128 COMMs carrying 1 KB each	52.3 ms	211 ms	602 ms	6.4 %
REV transfer through the SystemVault	81.6 ms	—	—	—

Table 3: Where the time of one envelope goes (`cost_breakdown_of_one_envelope`, `a_user_vault_transfer_costs`). "Interpret term" is an unmetered exploratory run of the same term. Spawning a runtime costs 3.6 ms and is subtracted.

under any budget. Work under the system signature is never drawn from a budget, so no allowance can bound it. Its event count is deterministic and linear in the number of sources: about 157 events (16 KB) per source per play.

6 Design: cost-accounted order-then-execute on REV stacks

6.1 Principles

P1. **Wallets are located REV stacks.** REV lives on its owners' channels, disaggregated from the vault. Moving REV is a native operation on the payer's and payee's channels.

P2. **Phlo is what a COMM spends.** Phlo is minted per deployment against REV in escrow, at the deployment's stated price. The reducer pays for a COMM by spending phlo.

P3. **Linearity.** REV changes hands and is never created or destroyed by computation. Consumed REV moves to the counterparty that the term names, which is the block's proposer.

P4. **No shared state on a deploy's payment path.** Everything a deploy reads or writes to pay for itself is named by its own signatures: its wallets, its funding slots, its escrow. The one exception is the proposer's REV channel, where deposits commute.

P5. **Concurrency from independence.** Envelopes whose footprints are Bernstein-independent run concurrently. Conflicts are races on shared data channels, decided by the runner and recorded.

P6. **The consensus seam stays.** Order-then-execute, structural validity, the linearizer, records, attestation, inclusion checks, the finalizer design and backpressure are those of revision 2, as delivered in patches 0028–0035.

6.2 Price and acceptance

A deployment states `phloLimit`, the most phlo it may spend, and `phloPrice`, the REV it pays per phlo. These keep their meaning from `dev`, as D-12 requires.

- **The shard floor is a validity rule.** A block that includes a deployment priced below the shard's minimum price is invalid (block-validity step 8, preserved by D-12). The floor sits in the genesis policy.

- **The validator's price is its own.** Above the floor, each validator sets the price at which it accepts deployments, as a node setting. A deployment below a validator's price is simply not included by that validator. This is a market, not a consensus rule.

6.3 The life of an envelope

```
gate(e, R):                          -- at the block pre-state, before any step
  Delta = linear demand of e in phlo (Delta_max where data-dependent), <= phloLimit
  for each wallet w that e's signers fund it from:
    need_w  = w's share of Delta * phloPrice
    have_w  = REV on w's located stack at R          (native read)
  accept iff have_w >= need_w for every w           (one local proof per purse)
precharge(e):                        -- in block order
  move need_w REV from each wallet w into escrow(e)  (native)
  mint Delta phlo cells against escrow(e), at phloPrice
  a wallet is short (an earlier envelope drew on it): reject e, no change
run(e):                              -- concurrent with the other envelopes
  each COMM spends phlo from escrow(e) under rules 1-5 and the join schema
settle(e):                           -- in block order, after the merge
  REV of the k phlo spent      -> REV channel of sender(block)
  REV of the Delta - k unspent -> the wallets it came from
  exhausted (signed-limit bound reached): all Delta * phloPrice -> sender(block)
```

- **Native, not Rholang.** The wallet read, the escrow moves, the phlo mint and the settlement are RSpace operations on signature channels, done by the runtime. They run no contract code and leave no Rholang event log. Their cost is a few produce and consume operations per wallet.
- **The escrow is the reservation.** REV in escrow is no longer in the wallet, so a later envelope's gate sees the reduced balance. Duplicate and simultaneous arrivals behave as the papers describe.
- **Phlo does not outlive its deployment.** Phlo cells exist only between precharge and settlement. Every cell ends either spent, its REV moved to the proposer, or unspent, its REV returned to the wallet.
- **The overestimate tightens the prepayment.** With a complete proof the precharge is $\Delta \times$ `phloPrice`, often far below `phloLimit` $\times$ `phloPrice`. With an incomplete proof it is the signed limit, exactly as in `dev`.

6.4 REV transfers

A transfer from wallet A to wallet B is a deployment signed by A. Its few COMMs spend phlo like any other deployment. The movement of REV is one native debit of A's stack and one native deposit on B's channel. Deposits land as separate datums, so transfers to one payee never conflict, and transfers between disjoint pairs run concurrently. Only spends from one wallet form a chain, and the linear proof already orders those.

On v6 the SystemVault stops being the custodian of REV. At most it remains as a compatibility facade: `find` returns the wallet channel, and `transfer` is the native transfer. Legacy shards keep the vault unchanged.

6.5 Supply

Genesis creates the wallets as located stacks with their initial balances. Only mint authority creates REV: genesis, and the PoS reward and epoch rules. Every other operation conserves it, and every block's executor checks conservation as an invariant.

6.6 Owners, sponsors and grants

- **Owners** (DR-135) pay for the regions of their stored processes at their own price. An owner holds a REV wallet, and phlo is minted against it per region at the owner's price. An owner without REV refuses the introduction, as the branch's region refusal already does.
- **Sponsors** use funding slots: an unforgeable channel holding deposited REV, named by the deploy that it funds.
- **Grants** are funding slots with a validity window that the gate checks at block time.

6.7 Block execution

1. **Gate** every envelope at R_0. The checks are independent per wallet and run concurrently.
2. **Precharge** in block order, natively. Envelopes that draw on one wallet are serialized here by that wallet's balance, exactly as the linear proof prescribes. Different wallets' precharges touch different channels.
3. **Run** every admitted envelope concurrently from the precharged root.
4. **Merge** in block order. A conflicting envelope is rolled back and keeps its charge for the run, as decided on October 25.
5. **Settle** in block order, natively: REV to the proposer and back to the wallets.

The ordered steps (2 and 5) are cheap native operations. The expensive step, the run, is the concurrent one.

6.8 Records

A record entry carries the gate verdict, the outcome, the phlo minted and spent, the REV moved per wallet and to the proposer, the budget recording and operation journal needed for replay, and the user COMM log. It carries no system log. With the 0035a analysis of 1.25 bytes of user log per phlo (0.1 measured), an entry is at most $1.25\,\Delta + r_0$ bytes, with r_0 under 4 KB measured. A per-block demand cap D with $1.25\,D + E\,r_0 \leq$ `block-record-bytes` then guarantees that every record can be cited. That closes the liveness hole of 0035 by arithmetic alone, and citing records in pieces (0035b) is no longer needed.

6.9 What leaves the v6 path

- The SystemVault as custodian of REV, its tree map of purses, its registry lookup and its mint-based purse split.

- The settlement call `applyPhloCost`, its unlimited budget, its burn of the charge, and its logs in the record.
- Vault reads in the gate. Supply becomes a wallet balance read natively.
- Holds as fit checks, `decide_groups`, `LaterHolds` and the held-capacity guard.
- The DR-28 monetary cursors and their vault validation, and with them the reason for the DR-119 bucket locks.
- The shared fee vault and the fixed fee.
- The vault-based precharge and refund of October 28 (withdrawn before merge).

All of these stay on legacy shards, which remain byte-identical to `dev`.

6.10 The consensus seam (unchanged from revision 2)

The following stand as designed in revision 2 and delivered in patches 0028–0035:

- **Block content and validity.** Accepted envelopes without event logs, an attestation (E, r_E), cited records, and structural validity that runs no Rholang and replays nothing.
- **Canonical order.** Epochs along the main chain, a topological sort with fixed tie-breaks, and A-LIN checked by the linearizer and at commit.
- **Executor, records and attestation.** One runner, verified replay, canonical records, attestation checks and evidence.
- **Pruning, lag gate and pacing.** Patches 0030, 0032 and 0035.
- **Charge model.** Charges as functions of events, operations named by source and occurrence, recorded interleaving decisions, concurrent replay with a bounded verdict (B2).
- **Finalizer.** Compute, Validate and Commit (A1a–A1c).
- **Epochs and slashing.** Bonds from attested state with an epoch lag (B6).

7 Throughput analysis

7.1 Model

Let an envelope's core time be the sum of four parts: its interpretation u, the metering overhead factor m on that interpretation, its funding work f (wallet read, escrow moves, phlo mint and settlement), and its share c of the block's checkpoint and merge. On P cores with parallel efficiency η, and with the longest chain L short relative to the work, node throughput is

$$X \;\approx\; \frac{\eta\, P}{m\, u + f + c}.$$

Every validator executes or replays every block, so shard throughput equals the throughput of one node. Adding validators adds safety, not throughput.

Today. f is the run-phase funding overhead (110–160 ms) plus settlement (560–600 ms), and settlement is one chain. The parallel term does not apply to it, and its per-envelope cost grows with block size. A REV transfer adds 81.6 ms of vault contract code on top.

Design. $m = 1.2$, from the branch's own profiles (15–23 % more CPU in the reducer). The funding work f covers a native wallet read, one escrow move each way, the phlo mint, the settlement moves and the gate's linear estimate. The checkpoint share is $c \approx 0.5$ ms (31 ms per 64-envelope block measured). Together $f + c$ is taken as 2 to 8 ms, with 5 ms as the middle value. A REV transfer adds one native debit and one native deposit, taken as 0.5 to 2 ms. These ranges are estimates until S1 measures them.

7.2 Per-envelope work

Workload	Today	Design (range)	Gain, middle	Gain, range
One send	675 ms	6.4 ms (3.4–9.4)	105×	72–198×
REV transfer	756 ms	7.4 ms (3.9–11.4)	102×	66–194×
256 sends	796 ms	40.6 ms (37.6–43.6)	20×	18–21×
300 recursive COMMs	756 ms	44.0 ms (41.0–47.0)	17×	16–18×
128 COMMs carrying 1 KB	813 ms	67.8 ms (64.8–70.8)	12×	11–13×

Table 4: Core time per envelope, today (run phase plus settlement, single-envelope block; for the transfer, plus the measured 81.6 ms of vault code) and projected. A transfer's own COMMs are taken as a one-send deployment. The gain is largest where the deploy does least, because today's cost is almost all funding overhead.

7.3 Node throughput

Workload	2 cores	8 cores	16 cores	32 cores
One send	187–281	750–1,124	1,499–2,249	2,998–4,497
REV transfer	162–243	648–972	1,297–1,945	2,593–3,890
256 sends	30–44	118–177	236–354	472–709
300 recursive COMMs	27–41	109–164	218–327	436–655
128 COMMs carrying 1 KB	18–27	71–106	142–213	283–425

Table 5: Projected envelopes per second for one node, with parallel efficiency η from 0.6 to 0.9, for envelopes with independent signers and no shared data channel. Today's measured rate on the same 2-core host is 0.44 (64-envelope blocks) to 1.24 (single-envelope blocks). Absolute values are for the measurement host. Faster hosts scale all figures alike, today's included.

Reading the tables. On 2 cores, the design projects 150–640 times today's measured rate for one-line deploys, and 130–550 times for REV transfers, against 1.24 envelopes per second for single-envelope blocks and 0.44 for 64-envelope blocks. For the heavier workloads it projects 14–36 times their single-envelope rate of 1.2–1.3 per second. On 16 cores, with independent signers, the factors become about 1,050–5,100 for small deploys and transfers, and 110–290 for the heavier ones. The gain has two sources that multiply: removing fixed funding overhead and the vault (Table 4), and replacing one causal chain with an antichain, so that cores apply (Table 5).

7.4 Where the anticipated gains do not appear

The model supports large gains for overhead, payments and concurrency. It does not support them unconditionally. These are the cases where it projects less, and why.

1. **Deploys dominated by their own computation.** The gain is $(m\,u + f_{\text{today}})/(m\,u + f)$. For a deploy that interprets for 50 ms it is about 12, and it falls toward 1 for very heavy deploys. Against the unmetered legacy path it falls toward $1/m$, below 1. The design removes overhead. It does not make Rholang faster, and the metering factor $m = 1.2$ is a real cost. The interpreter's own defects (clone audit, Tier 0: quadratic `Env::put`, quadratic replay scan, quadratic `eval_match`) are separate wins of 2.7–273 times on the patterns they affect.
2. **Application hot channels.** Two deploys that write the same data channel, such as a popular contract's state or a liquidity pool kept in one contract, form a chain. If a fraction h of the work lies on one chain, Amdahl's law limits the speedup on P cores to $1/(h + (1 - h)/P)$. On 16 cores, h of 1, 5, 10 and 25 % gives 13.9, 9.1, 6.4 and 3.4. The design removes the system's hot spots, including the vault. Applications can still create their own.
3. **One busy wallet.** All spends from one wallet form a chain, by the papers' own result. A single account that submits most of the load is serialized at its precharge. The precharge is cheap, but the order is fixed. Deposits to one wallet do not have this limit, because they commute.
4. **The consensus cap.** A block carries at most $E = 256$ envelopes, and blocks execute in canonical order. Shard throughput is at most $E \cdot V \cdot r$ for V validators proposing r blocks per second: 768 per second for three validators at one block per second, 1,536 for six. For small deploys and transfers this cap binds before 16 cores do. Raising it means a larger E (bounded by the record arithmetic above), faster proposals, or executing blocks with disjoint footprints concurrently (refinement R1).
5. **Replication.** Every node executes or replays every block, so validators add no throughput. Replay is cheaper than execution, with no matching search, but still proportional to the work.
6. **Multi-core efficiency is not measured.** The range $\eta = 0.6$–0.9 is an assumption. On today's code, the run phase's core time per envelope grows from 140 ms alone to 254 ms in a 64-envelope block on 2 cores. Part of that is vault reads in the gate, which the design removes, and part may be contention in RSpace's hot store. Until S7 measures it, the 8- to 32-core columns are projections, not results.
7. **The funding work estimate.** The 2–8 ms for native funding work and the checkpoint share is an estimate from RSpace operation costs. If native stack operations cost as much as Rholang COMMs (about 1 ms each, measured in the legacy reducer breakdown), f approaches the top of its range and the one-send gain falls toward 72 times.

7.5 Validation

These measurements turn the projections into exit criteria:

- **S1 microbenchmark.** The gate, precharge, phlo mint and settlement for one envelope funded from one to four wallets. Target: under 8 ms on the measurement host.
- **S3 transfer benchmark.** A REV transfer between two wallets as a deployment, end to end. Target: under 12 ms per transfer on the measurement host, and at least 150 transfers per

second on 2 cores with independent payers.

- **S7 block benchmark.** `phase_timings_by_block_size` and `cost_breakdown_of_one_envelope` rerun on the new path. Targets: settlement under 5 % of block time, settlement per envelope flat in block size (within 20 % from 1 to 64 envelopes), and one-send throughput of at least 150 per second on 2 cores.
- **S7 scaling run.** The same block benchmark on 8 and 16 cores. Target: $\eta \geq 0.6$. A lower result points to RSpace contention and opens a work item.
- **Conservation.** Every executor and benchmark run checks that the sum over wallets, escrows and validator channels is unchanged by each block.
- **B7 soak.** Sustained throughput against a legacy soak on the same hardware, with the hot-channel workloads of item 2 above as a separate profile.

8 Development plan

8.1 Status (October 29, 2026)

Work	State
Track A (A1a–A9)	Delivered on `dev`: the oracle cache, the timed derivation, the Compute/Validate/Commit finalizer, coalesced proposals, pacing and admission cap, pool, register, runtime and DAG fixes, and the capped rejection search (off by default).
B4, patches 0028–0035	Delivered on the integration branch: contracts and persistent executor state, the v6 body field, the execution-lag gate, deferred inclusion checks, the A-LIN switch, pruning, and configuration, record packing and pacing.
0035a, vault metering and the vault-based precharge	Withdrawn. The P1 experiment is not merged, and its measurements are in §5.

8.2 Track B, revised: REV stacks and native settlement

The series S1–S7 replaces the old 0035a scope and the settlement part of B3.

ID	Change	Exit criterion	Size
S1	Native escrow and phlo: precharge from wallet stacks into an envelope escrow, phlo minted at the deployment's price, the run spending phlo, settlement of spent REV to the proposer's channel and unspent REV to the wallets. Built on the branch's `CostStack` datums and native stack accounting. Conservation as an executor invariant.	Conservation tests; replay equality; S1 microbenchmark under 8 ms.	M
S2	Gate from wallet balances read natively, and the price checks: the shard floor in validity, the validator's acceptance price as a node setting. Remove the vault snapshot reads from the v6 gate.	Gate time independent of vault state; the gate property tests pass.	S–M
S3	Wallets as located REV stacks: genesis creates them on v6 shards; the native transfer; deposits as separate datums combined on spend; multi-signature and contract-held wallets; the SystemVault facade; client tooling (`node deploy`, Embers, the gateway).	S3 transfer benchmark; a REV transfer touches no vault structure.	M–L
S4	Owners on REV wallets with phlo minted per region; sponsors and grants on funding slots.	The DR-135 owner tests on v6; sponsored and granted deploys.	M
S5	Records without system logs, and the record-size coupling (the demand cap and its genesis check). 0035b is dropped.	A 256-envelope block's record always fits; the cross-host fixture is regenerated.	S
S6	Remove from the v6 path: `applyPhloCost` and its burn, the cursors, `LaterHolds`, `decide_groups`, the fee vault and the fixed fee. Legacy paths unchanged.	No vault call on the v6 path.	S
S7	Throughput benchmarks and targets (§7.5) as B3 exit criteria.	The targets met, or a measured explanation.	S

Sizes: S is up to 1 engineer-week, M is 1–3, L is 3–8. S1 and S2 are the critical path, then S3. S4 can run beside S5 and S6. With two engineers the series takes about six to eight weeks, and S7 reports the first throughput numbers at the end of S2 and the transfer numbers at the end of S3.

8.3 Other tracks

- **B5–B7** stand as in revision 2 (records and evidence, epochs, the soak and activation), with the throughput targets added to B7.
- **Interpreter defects** (clone audit, Tier 0): quadratic environment copying, the quadratic replay

scan, quadratic match fall-through and quadratic stream reassembly. These are independent of cost accounting, low risk, and they help every path. They should land on `dev` beside Track A.

- **The D-12 mapping.** `phloLimit` and `phloPrice` keep their `dev` meaning under this design (§6). The mapping is written up as the normative text D-12 asked for, and ratified before S2 lands.

9 Decisions, risks and open questions

9.1 Decisions requested

	Decision	Proposed
1	Wallets are located REV stacks on their owners' channels, disaggregated from the vault. Moving REV is native.	Yes. The vault stops being the custodian of REV on v6.
2	Phlo is minted per deployment against REV in escrow, at the deployment's stated price.	Yes.
3	The shard minimum price is a validity floor; each validator sets its own acceptance price above it.	Yes.
4	Consumed REV moves to the proposer that the block names. No burn, no fixed fee apart from phlo.	Yes.
5	Owners on REV wallets; sponsors and grants on funding slots.	Yes.
6	The throughput targets of §7.5 become exit criteria.	Yes.
7	The clone-audit Tier 0 fixes land on `dev` now.	Yes.

9.2 Risks

Risk	Mitigation
The branch's stack machinery is less complete than it appears	S1 starts with a survey of `CostStack`, `PurseInventory` and the native stack accounting, and reports gaps before building on them.
Multi-core efficiency is low (RSpace contention)	The S7 scaling run measures it early. RSpace hot-store contention becomes a work item if $\eta < 0.6$.
Moving balances out of the vault at v6 activation	Genesis creates wallets as stacks on new v6 shards. A migration for existing balances is a separate decision (Q-15).
Client changes	The deployment contract (`phloLimit`, `phloPrice`) is unchanged. Wallet queries and transfers change behind the vault facade; tooling changes are part of S3.
Applications build their own hot channels	Document the chain rule for contract authors, and that deposits commute while spends from one wallet do not.

Risk	Mitigation
The consensus cap binds before cores do	Raise E within the record arithmetic, tune the proposal rate, and pursue refinement R1.
Conservation broken by a defect	Conservation is an executor invariant checked on every block, in play and replay.
Risks of revision 2	A-LIN, the finalizer commit, the charge model and host-work determinism stand as listed there and are mitigated as delivered.

9.3 Open questions

Q-1 to Q-12 of revision 2 stand. Q-2 (evidence penalties) and Q-7 (fallback distance, record cap, retention) have been decided. Q-13 of revision 3 (the mapping of `phloLimit` and `phloPrice`) is answered by §6. Open:

ID	Question
Q-14	How the PoS reward and epoch rules mint REV, and whether REV received on a proposer's channel is shared with its delegators or stakers, and when.
Q-15	How existing REV balances move to stacks if an existing shard activates v6: at genesis of a new v6 shard only, or by a migration.
Q-16	Whether the escrow lives as a datum on an envelope-keyed channel or in the envelope's runtime state, given that replay must reproduce it.
Q-17	The form of a validator's acceptance price: one price, or a schedule by resource class.

A Detailed stall findings

The table lists the findings of the stall review beyond those in Section 3, by subsystem, with the lines at which each was located on `master` at `7264c98`. Severity is the review's estimate of the contribution to a stall that worsens under load.

Subsystem	Finding	Location	Sev.
Proposer	Unfinalized cone walked once per parent with full block decodes.	`block_creator.rs:1497-1725, 2085-2131`	High
Proposer	Whole pool sorted and encoded per selection, three times.	`block_creator.rs: 269-279,349-415`	Med
Proposer	Per-deploy WARN logged for every in-scope deploy on every proposal.	`block_creator.rs:1380-1440`	Med
Merge	Retry loop rebuilds branches and conflict map each pass.	`dag_merger.rs: 705-752`	High

Subsystem	Finding	Location	Sev.
Merge	Stale-diff expansion runs one DAG search per block pair.	`dag_merger.rs:1855-1915`	Med
Merge	Dependency ordering cubic in branch size; `depends` allocates sets per call.	`dag_merger.rs:82-120,1456-1502`	Med
Merge	`ChannelChange::combine` quadratic on hot channels.	`channel_change.rs:17-55`	Med
Merge	Block-index cache returns deep clones; bypasses its FIFO bound.	`interpreter_util.rs:1316-1342`	Med
Finality	Uncached LMDB metadata lookups inside `block_in_place`.	`block_dag_key_value_storage.rs:144-151`	High
Finality	FT propagation rescans up to 50,000 blocks under both write locks.	`block_dag_key_value_storage.rs:1699-1727`	Med
Finality	Estimator scoring walks $O(V \times (\text{tip} - \text{LCA}))$.	`estimator.rs:232-378`	Med
Finality	Redundant fault-tolerance oracle run after each LFB advance.	`finalization_runner.rs:571-576`	Med
Block processing	DAG membership via full representation per gossip message.	`block_admission.rs:34-67`	High
Block processing	Pendant full scan every 32 blocks with block decodes.	`buffer_resolver.rs:59-109`	Med
Block processing	Retriever sends serially, one hash at a time.	`block_retriever.rs:1032-1139`	Med
Block processing	Full snapshot computed per incoming block; cache keyed on DAG generation.	`block_processor.rs:548-553`	Med
RSpace	`put_continuation` formats a `Debug` string per waiting continuation.	`hot_store.rs:528-559,1000`	Med
RSpace	Checkpoint CPU and LMDB writes on tokio workers.	`ispace_impl.rs:41-78`	Med
RSpace	History caches cleared in bulk; radix cache evicts arbitrary nodes.	`hot_store.rs:1091-1117`, `radix_tree.rs:875-890`	Med
Node	gRPC `find_deploy` server-side retry loop.	`deploy_grpc_service_v1.rs:570-643`	Med
Node	HTTP offload without an admission limit.	`shared_handlers.rs:626-640`	Low

B Sources

Papers, read directly, in `F1R3FLY-io/publications` at `37915af`:

- `cost-accounting/cost-accounted-rho.tex`: signatures as names, token stacks, the five rules, the join schema, object capabilities and located stacks, funding slots, the RSpace and Casper implications, the transaction analysis, the linear resource proof, overcharge and refund, the acceptance protocol, deployment boundaries, the translation to pure rho, and the validator fee example.
- `cost-accounting-as-monad/continued-gslt-cost-v2.tex`: the cost endofunctor, the two monoids, located resource stacks, and local sufficiency of per-purse proofs.
- `cost-spacetime/spacetime-functor.tex`: footprints and Bernstein independence; time from the stack and space from the surfaces; number as volume; proper time as the longest chain.
- `virtual-tokens/virtual-token.tex`: the virtual token as a unit of account.
- `history-and-cost/two-erasures.tex`: internalisation as a lax algebra with an overhead grade.
- `f1r3node-profiling/clone-audit.tex`: the interpreter's asymptotic copy defects and their microbenchmarks.
- `cost-accounting/stall-loop-design-5-10-1-response.tex`: the Casper Working Group's finalizer design.

Code, `F1R3FLY-io/f1r3node-rust`, branch `integration/cost-accounted-ote-b1d866e45` at `d1f3ab5e1`:

- in `rholang/src/rust/interpreter/`: `reduce.rs`, `accounting/authority.rs` and `accounting/delta_sigma/estimate.rs`;
- in `casper/src/rust/util/rholang/costacc/`: `supply.rs`, `owner_offer.rs` and `direct_wallet_funding/execution/`;
- `casper/src/main/resources/SystemVaultV6.rho` and `casper/src/rust/v6/`.

Measurements, on the host named on the first page: `v6_executor_spec::phase_timings_by_block_size`, `cost_breakdown_of_one_envelope`, `a_user_vault_transfer_costs`, the record measurements of patch 0035, the vault-play measurements of October 28, and `cost_accounting_perf_spec` (legacy precharge, refund and replay).

Earlier designs: revisions 1 and 2 of this document (October 16, 2026); D-12 (deploy cost limits, rejected September 16, 2026).